Rajarata University of Sri Lanka
Department of Computing

ICT1402
PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 12

Working with Functions

K.A.S.H. Kulathilake
Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
  - Describe the C functions.
  - Practice the declaration, parameter passing, prototyping and calling functions.
  - Justify parameter passing methods.
  - Practice creating user defined C header files.
  - Justify the variable types and scope rules.
  - Describe the concept of recursion.
  - Apply taught concepts for writing programs.

Introduction

- A function groups a number of program statements into a unit and gives it a name.
- This unit can then be invoked from other parts of the program.
- We can make use of existing functions available in the various C libraries.
- The standard library provides a rich collection of functions.
- We can also write our own functions.

Why Functions?

- Modular programming
- Reduction in amount of work and development time
- Program and function debugging is easier
- Reduction in size of the program due to code reusability
- Functions can be accessed repeatedly without redevelopment

Function Definition

```
return_type function_name (type1 name1,
type2 name2,...,typen namen)
{
      statement 1;
      statement 2;
      …….
       statement n;
      return value;
}
```

Statements can be either declaration statements or executable statements. Declaration statements must be stated at first within the function body.

Return value must be equal to the return type of the function.
You can return value or initialized variable as the return value.
Any value is not returned from the function if the return type has been declared as “void”.

Function Definition (Cont…)

```
void printName ()
{
      printf("My name is Saman \n");
}

double calculateSale ()
{
      int qty =10;
      double unitPrice = 10.
      double total = 0;
      total = qty * unitPrice;
      return total;
}
```

Return data type can represent any of C data types, such as char, float, double and all of the different int types. Function name must be unique within the program.

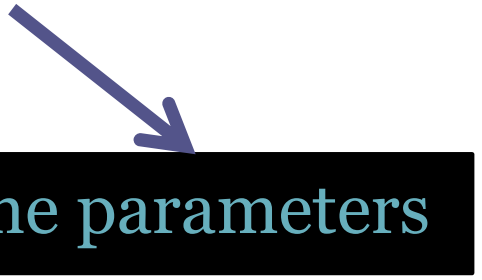
A function may or may not have an argument list, depending on whether the called function needs to receive data from the calling function.

Function Definition (Cont…)

```
double calculateSale ( int qty, double
unitPrice)
{
     double total = 0;
     total = qty * unitPrice;
     return total;
}
```

Function Prototype

return_type function_name (type1 , type2,...,typen);



e.g.

```
void printName ();
double calculateSale ();
double calculateSale ( int , double);
```

Following statement is also correct:

```
double calculateSale ( int qty, double
unitPrice);
```

Function Prototype (Cont…)

Why function prototyping is essential?

The main reason they exist is so that the compiler can go through the program exactly once, knowing which **functions** are in scope at any given time. To do this, **functions** would have to be declared before they're used; **prototypes** let you declare a **function** without implementing its body until later.

Calling a Function

- While creating a C function, you give a definition of what the function has to do.
- To use a function, you will have to call that function to perform the defined task.
- When a program calls a function, the program control is transferred to the called function.
- A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

Calling a Function (Cont…)

To call a function, you simply need to pass the required parameters along with the function name, and if the function returns a value, then you can store the returned value.

Calling a Function (Cont…)

- The C compiler assumes that a function returns a value of type int as the default case.
- More specifically, when a call is made to a function, the compiler assumes that the function returns a value of type int unless either of the following has occurred:
  - The function has been defined in the program before the function call is encountered.
  - The value returned by the function has been *declared* before the function call is encountered.

Parameter Passing

```
#include <stdio.h>

double calculateSale (int, double);

double calculateSale (int qty, double
unitPrice)
{
     double total = 0;
     total = qty * unitPrice;
     return total;
}
```

Parameter Passing (Cont…)

Passing Constants as Arguments:

```
int main ()
{
    printf ("Total sale is : %0.2f \n",
                calculateSale (10, 10.50));
    return 0;
}
```

Parameter Passing (Cont…)

Can’t we pass constants and variables as argument for a function?

Parameter Passing (Cont…)

Passing Constants as Arguments:

```
int main ()
{
    double totalSale = 0;
    totalSale = calculateSale (10, 10.50);
    printf ("Total sale is : %0.2f \n",
                                totalSale);
    return 0;
}
```

Parameter Passing (Cont…)

Passing Variables as Arguments:

```
int main ()
{
     int units = 10;
     double unitCost = 10.50;
     double totalSale = 0;
     totalSale = caculateSale (units,
                               unitCost);
     printf ("Total sale is : %0.2f \n",
                                totalSale);
     return 0;
}
```

Demonstration

```
#include<stdio.h>

void astrixChart (int, char);

void astrixChart (int number, char style)
{
     int x;
     for (x = 0; x < number; x++)
          printf("%c", style);
     printf("\n");
}
```

Demonstration (Cont…)

```
int main()
{
      int m1, m2;
      printf("Enter percentage score of Group1 and
                                                Group2 : ");
      scanf ("%i %i",&m1, &m2);

      printf("Group1 :" );
      astrixChart (m1, '*');

      printf("Group2 :" );
      astrixChart (m2, '#');

      return 0;
}
```

How Many Return Statements Per Function?

```
#include <stdio.h>
int ageState (int);
int ageState (int age)
{
        if (age <18)
                return 0;
        else
                return 1;
}

int main ()
{
        int age =18;

        if (ageState (age) == 0)
                printf("Child \n");
        else
                printf("Adult\n");

        return 0;
}
```

Advantages of Functions

- Functions help *avoid duplication* of effort and code in programs.
  - During the development of a program, the same or similar activity may be required to be performed more than once.
  - The use of functions in such situations avoids duplication of effort and code in programs.
  - This reduces the size of the source program as well as the executable program.
  - It also reduces the time required to write, test, debug and maintain such programs, thus reducing program development and maintenance cost.

Advantages of Functions (Cont…)

- Functions allow the ***divide and conquer* strategy** to be used for the development of programs.
  - When developing even a moderately sized program, it is very difficult if not impossible, to write the entire program as a single large main function.
  - Such programs are very difficult to test, debug and maintain.
  - The task to be performed is normally divided into several independent subtasks, thereby reducing the overall complexity; a separate function is written for each subtask.
  - In fact, we can further divide each subtask into smaller subtasks, further reducing the complexity.

Advantages of Functions (Cont…)

- Functions enable us to *hide the implementation details* of a program, e. g., we have used library functions such as sqrt, log, sin, etc. without ever knowing how they are implemented.
  - However, although we need to know the implementation details for user-defined functions, once a function is developed and tested, we can continue to use it without going into its implementation details.
  - Another consequence of hiding implementation details is improvement in the readability of a program.
  - Proper use of functions leads to programs that are easier to read and understand.

Advantages of Functions (Cont…)

- The functions developed for one program can be used, if required, in another with little or no modification. This further reduces program development time and cost.

Parameter Passing Methods

- If a function is to use arguments, it must declare variables that accept the values of the arguments.
- These variables are called the **formal parameters** of the function.
- Formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.
- While calling a function, there are two ways in which arguments can be passed to a function
  - Pass by value
  - Pass by address/ reference

Parameter Passing Methods (Cont…)

- Pass by Value:
  - This method copies the actual value of an argument into the formal parameter of the function.
  - In this case, changes made to the parameter inside the function have no effect on the argument.
  - By default, C programming uses *call by value* to pass arguments.
  - In general, it means the code within a function cannot alter the arguments used to call the function.

Parameter Passing Methods (Cont…)

```
/* function definition to swap the values */
void swap(int x, int y) {

   int temp;

   temp = x; /* save the value of x */
   x = y;    /* put y into x */
   y = temp; /* put temp into y */

   return;
}
```

Parameter Passing Methods (Cont…)

```
#include <stdio.h>
/* function declaration */
void swap(int x, int y);
int main () {
   /* local variable definition */
   int a = 100;
   int b = 200;
   printf("Before swap, value of a : %d\n", a );
   printf("Before swap, value of b : %d\n", b );
   /* calling a function to swap the values */
   swap(a, b);
   printf("After swap, value of a : %d\n", a );
   printf("After swap, value of b : %d\n", b );
   return 0;
}
```

It shows that there are no changes in the values, though they had been changed inside the function.

Parameter Passing Methods (Cont…)

- Pass by Reference
  - This method copies the address of an argument into the formal parameter.
  - Inside the function, the address is used to access the actual argument used in the call.
  - This means that changes made to the parameter affect the argument.
  - To pass a value by reference, argument pointers are passed to the functions just like any other value.
  - So accordingly you need to declare the function parameters as pointer types.

Parameter Passing Methods (Cont…)

```
/* function definition to swap the values */
void swap(int *x, int *y) {

   int temp;
   temp = *x;      /* save the value at address x */
   *x = *y;        /* put y into x */
   *y = temp;      /* put temp into y */

   return;
}
```

Parameter Passing Methods (Cont…)

```
#include <stdio.h>
/* function declaration */
void swap(int *x, int *y);
int main () {
   /* local variable definition */
   int a = 100;
   int b = 200;
   printf("Before swap, value of a : %d\n", a );
   printf("Before swap, value of b : %d\n", b );
   /* calling a function to swap the values.
      * &a indicates pointer to a ie. address of variable a and
      * &b indicates pointer to b ie. address of variable b.
   */
   swap(&a, &b);
   printf("After swap, value of a : %d\n", a );
   printf("After swap, value of b : %d\n", b );
   return 0;
}
```

It shows that the change has reflected outside the function as well, unlike call by value where the changes do not reflect outside the function.

Some Important Facts

- Here are some reminders and suggestions about functions:
  - Remember that, by default, the compiler assumes that a function returns an int.
  - When defining a function that returns an int, define it as such.
  - When defining a function that doesn't return a value, define it as void.
  - The compiler converts your arguments to agree with the ones the function expects only if you have previously defined or declared the function.
  - To play it safe, declare all functions in your program, even if they are defined before they are called. (You might decide later to move them somewhere else in your file or even to another file.)

Functions and Arrays

- As with ordinary variables and values, it is also possible to pass the value of an array element and even an entire array as an argument to a function.
- So, to take the square root of averages[i] and assign the result to a variable called sq_root_result, a statement such as

```
sq_root_result = squareRoot (averages[i]);
```

does the trick.

Functions and Arrays (Cont…)

- To pass an array to a function, it is only necessary to list the name of the array, without any subscripts, inside the call to the function.
- As an example, if you assume that gradeScores has been declared as an array containing 100 elements, the expression

```
minimum (gradeScores);
```

- in effect passes the entire 100 elements contained in the array gradeScores to the function called minimum.

Functions and Arrays (Cont…)

- Naturally, on the other side of the coin, the minimum function must be expecting an entire array to be passed as an argument and must make the appropriate formal parameter declaration.
- So the minimum function might look something like this:

```
int minimum (int values[100])
{
    ...
    return minValue;
}
```

Functions and Arrays (Cont…)

```
#include <stdio.h>
int minimum (int[]);

int minimum (int values[10])
{
     int minValue, i;
     minValue = values[0];
     for ( i = 1; i < 10; ++i )
          if ( values[i] < minValue )
               minValue = values[i];
     return minValue;
}
```

With your general-purpose minimum function in hand, you can use it to find the minimum of *any* array containing 10 integers.

Functions and Arrays (Cont…)

```
int main (void)
{
    int scores[10], i, minScore;
    printf ("Enter 10 scores\n");
    for ( i = 0; i < 10; ++i )
        scanf ("%i", &scores[i]);
    minScore = minimum (scores);
    printf ("\nMinimum score is %i\n",
                          minScore);
    return 0;
}
```

Functions and Arrays (Cont…)

- Modification
  - In the function declaration, you can omit the specification of the number of elements contained in the formal parameter array.
  - The C compiler actually ignores this part of the declaration anyway; all the compiler is concerned with is the fact that an array is expected as an argument to the function and not how many elements are in it.

Functions and Arrays (Cont…)

```
#include <stdio.h>

int minimum (int[], int);

int minimum (int values[], int numberOfElements)
{
      int minValue, i;
      minValue = values[0];
      for ( i = 1; i < numberOfElements; ++i )
            if ( values[i] < minValue )
                  minValue = values[i];
      return minValue;
}
```

Functions and Arrays (Cont…)

```
int main (void)
{
      int array1[5] = { 157, -28, -37, 26, 10 };
      int array2[7] = { 12, 45, 1, 10, 5, 3, 22 };

      printf ("array1 minimum: %i\n",
                  minimum (array1, 5));
      printf ("array2 minimum: %i\n",
                  minimum (array2, 7));
      return 0;
}
```

Variables & Scope

- A scope in any programming is a region of the program where a defined variable can have its existence and beyond that variable it cannot be accessed.
- There are three places where variables can be declared in C programming language –
  - Inside a function or a block which is called **local** variables.
  - Outside of all functions which is called **global** variables.
  - In the definition of function parameters which are called **formal** parameters.

Variables & Scope (Cont…)

- Local Variables/ Automatic Variables
  - Variables that are declared inside a function or block are called local variables.
  - They are automatically “created” each time the function is called, and because their values are local to the function.
  - They can be used only by statements that are inside that function or block of code.
  - Local variables are not known to functions outside their own.
  - If an initial value is given to a variable inside a function, that initial value is assigned to the variable *each* time the function is called.

Variables & Scope (Cont…)

```
#include <stdio.h>
int main () {
   /* local variable declaration */
   int a, b, c;
   /* actual initialization */
   a = 10;
   b = 20;
   c = a + b;
   printf ("value of a = %d, b = %d and
                        c = %d\n", a, b, c);
    return 0;
}
```

Here all the variables a, b, and c are local to main() function.

Variables & Scope (Cont…)

- Global Variables
  - Global variables are defined outside a function, usually on top of the program.
  - Global variables hold their values throughout the lifetime of your program and they can be accessed inside any of the functions defined for the program.
  - A global variable can be accessed by any function.
  - That is, a global variable is available for use throughout your entire program after its declaration.

Variables & Scope (Cont…)

```
#include <stdio.h>

/* global variable declaration */
int g;

int main () {
  /* local variable declaration */
  int a, b;
  /* actual initialization */
  a = 10;
  b = 20;
  g = a + b;
  printf ("value of a = %d, b = %d and g = %d\n", a, b, g);
  return 0;
}
```

Variables & Scope (Cont…)

- A program can have same name for local and global variables but the value of local variable inside a function will take preference.

```
#include <stdio.h>

/* global variable declaration */
int g = 20;

int main () {
  /* local variable declaration */
  int g = 10;
  printf ("value of g = %d\n",  g);
  return 0;
}
```

Variables & Scope (Cont…)

- Formal Parameters
  - Formal parameters, are treated as local variables with-in a function and they take precedence over global variables.

Variables & Scope (Cont…)

```
#include <stdio.h>
int sum(int,int);
/* global variable declaration */
int a = 20;
int main () {
  /* local variable declaration in main function */
  int a = 10;
  int b = 20;
  int c = 0;
  printf ("value of a in main() = %d\n",  a);
  c = sum( a, b);
  printf ("value of c in main() = %d\n",  c);
  return 0;
}
```

Variables & Scope (Cont…)

```
/* function to add two integers */
int sum(int a, int b) {

   printf ("value of a in sum() = %d\n",  a);
   printf ("value of b in sum() = %d\n",  b);

   return a + b;
}
```

Initializing Local & Global Variables

- When a local variable is defined, it is not initialized by the system, you must initialize it yourself.
- Global variables are initialized automatically by the system when you define them as follows –

Data Type	Initial Default Value
int	0
char	'\0'
float	0
double	0
pointer	NULL

Initializing Local & Global Variables (Cont…)

It is a good programming practice to initialize variables properly, otherwise your program may produce unexpected results, because uninitialized variables will take some garbage value already available at their memory location.

Header Files

- A header file in C programming language is a file with .h extension which contains a set of common function declarations and macro definitions which can be shared across multiple program files.
- C language provides a set of in build header files which contains commonly used utility functions and macros.
- **Types of Header Files in C**
  - User defined header files.
  - In-built header files.
- #inclide Preprocessor Directives is used to include both system header files and user defined header files in C Program.

Header Files (Cont…)

- **Syntax to Include Header File in C Program**

```
#include <Header_file_name>
```

- Above mentioned #include syntax is used to include in-built system header files.
- It searches given header file in a standard list of directories where all in-built header files are stored.
- To include in-built header file we use triangular bracket.

Header Files (Cont…)

- Above mentioned #include syntax is used to include user-defined system header files.
- It searches given user defined header file in a current directories where current c program exists.
- To include user-defined header file we use double quotes.

E.g.

```
#include <math.h> // Standard Header File #include
"myHeaderFile.h" // User Defined Header File
```

Create Your Own Header File

- Step1 : Type this Code

```
int add(int a,int b)
{
return(a+b);
}
```

- In this Code write only function definition as you write in General C Program
- Save this file with .h extension.
- Lets assume we saved this file as myhead.h.
- Copy myhead.h header file to the same directory where other inbuilt header files are stored.
- Compile this file.

  gcc –c myhead.h –o myhead.o

Create Your Own Header File (Cont…)

- To Include your new header file in a c program used #include preprocessor directive.

```
#include<stdio.h>
#include"myhead.h"

int main () {
    int num1 = 10, num2 = 10, num3;
    num3 = add(num1, num2);
    printf("Addition of Two numbers : %d", num3);
    return 0;
}
```

Questions

Triangular Number

```
#include <stdio.h>
void calculateTriangularNumber (int);
void calculateTriangularNumber (int n)
{
   int i, triangularNumber = 0;
   for ( i = 1; i <= n; ++i )
        triangularNumber += i;
   printf ("Triangular number %i is %i\n", n, triangularNumber);
}
int main (void)
{
   calculateTriangularNumber (10);
   calculateTriangularNumber (20);
   calculateTriangularNumber (50);
   return 0;
}
```

Questions

Greatest Common Divisor

```
#include <stdio.h>
void gcd (int, int);
void gcd (int u, int v)
{
        int temp;
        printf ("The gcd of %i and %i is ", u, v);
        while ( v != 0 ) {
                  temp = u % v;
                  u = v;
                  v = temp;
        }
        printf ("%i\n", u);
}
int main (void)
{
        gcd (150, 35);
        gcd (1026, 405);
        gcd (83, 240);
        return 0;
}
```

Questions

- Absolute Value

```
#include <stdio.h>
float absoluteValue (float);

float absoluteValue (float x)
{
     if ( x < 0 )
          x = -x;
     return x;
}
```

Questions

```
int main (void)
{
        float f1 = -15.5, f2 = 20.0, f3 = -5.0;
        int i1 = -716;
        float result;

        result = absoluteValue (f1);
        printf ("result = %.2f\n", result);
        printf ("f1 = %.2f\n", f1);
        result = absoluteValue (f2) + absoluteValue (f3);
        printf ("result = %.2f\n", result);
        result = absoluteValue ( (float) i1 );
        printf ("result = %.2f\n", result);
        result = absoluteValue (i1);
        printf ("result = %.2f\n", result);
        printf ("%.2f\n", absoluteValue (-6.0) / 4 );

        return 0;
}
```

Questions

- Make a small calculator program by using functions.
- Write following functions to calculate the area and circumference of a circle.
  - double calcArea (int radius);
  - double calcCircumference (int radius);

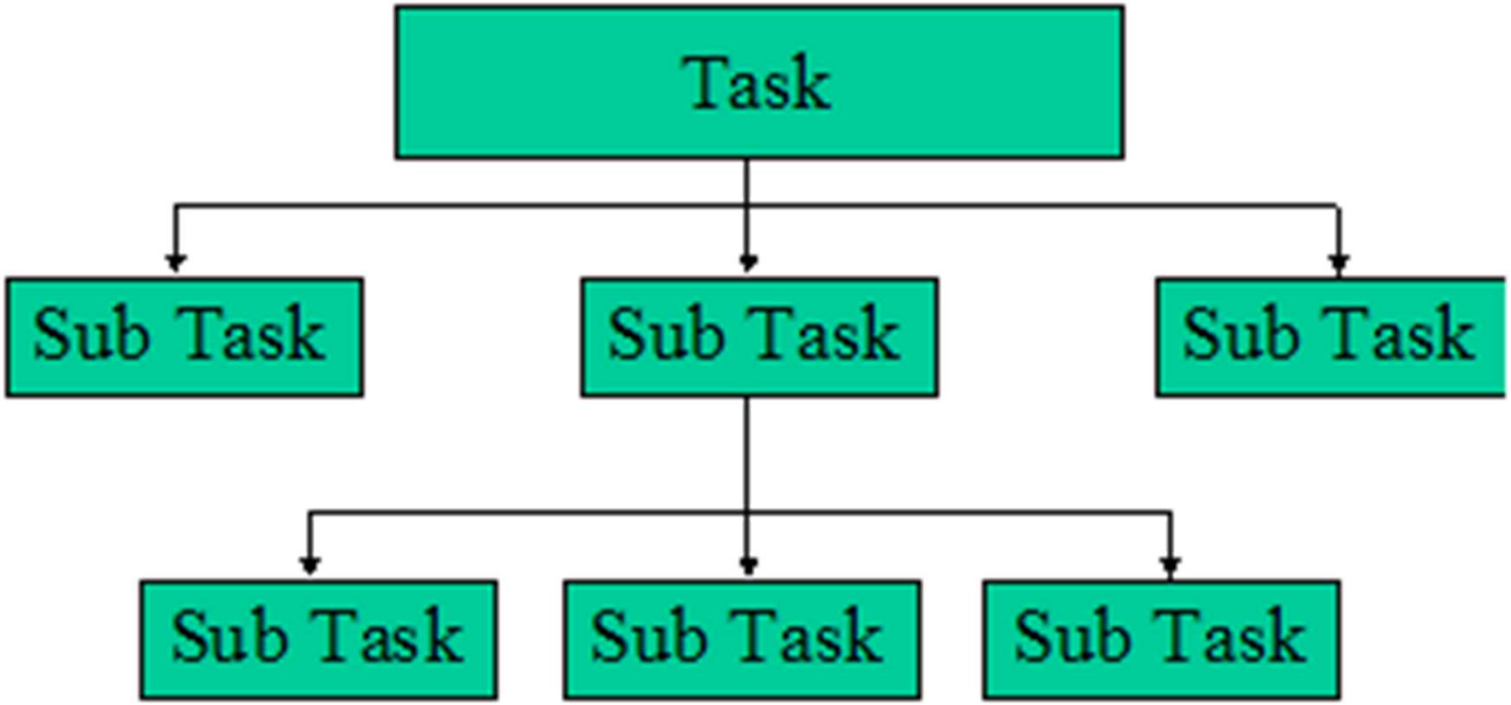
Questions

- Input employee number, name and basic salary of an employee and calculate monthly payment by using a function. Gross salary is calculated according to following: (note: HRA and DA are allowances).
  - Basic Salary <= 10000 : HRA = 20%, DA = 80%
  - Basic Salary <= 20000 : HRA = 25%, DA = 90%
  - Basic Salary > 20000 : HRA = 30%, DA = 95%
- Moreover, write another function to display calculated pay sheet.

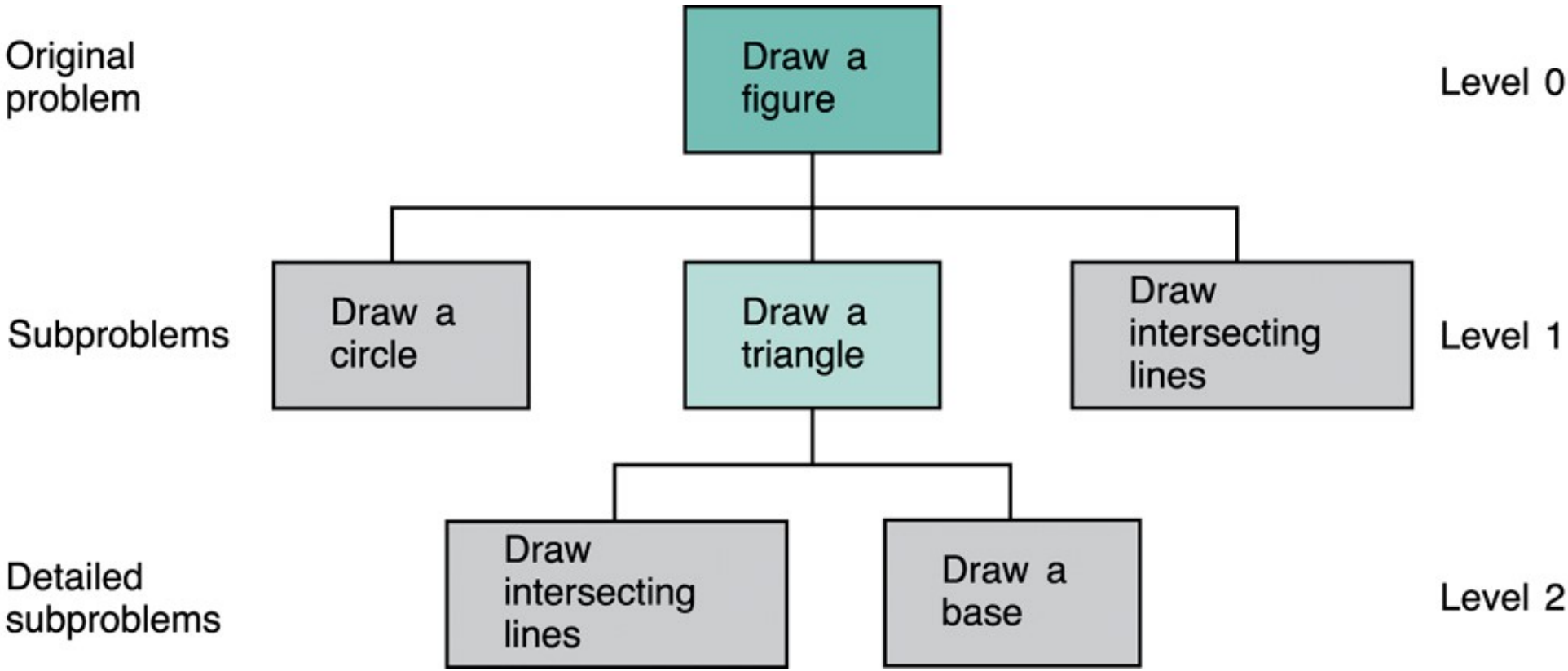
Top-Down Programming

- The notion of functions that call functions that in turn call functions, and so on, forms the basis for producing good, structured programs.
- Top-down programming refers to a style of programming where an application is constructed starting with a high-level description of what it is supposed to do, and breaking the specification down into simpler and simpler pieces, until a level has been reached that corresponds to the primitives of the programming language to be used.

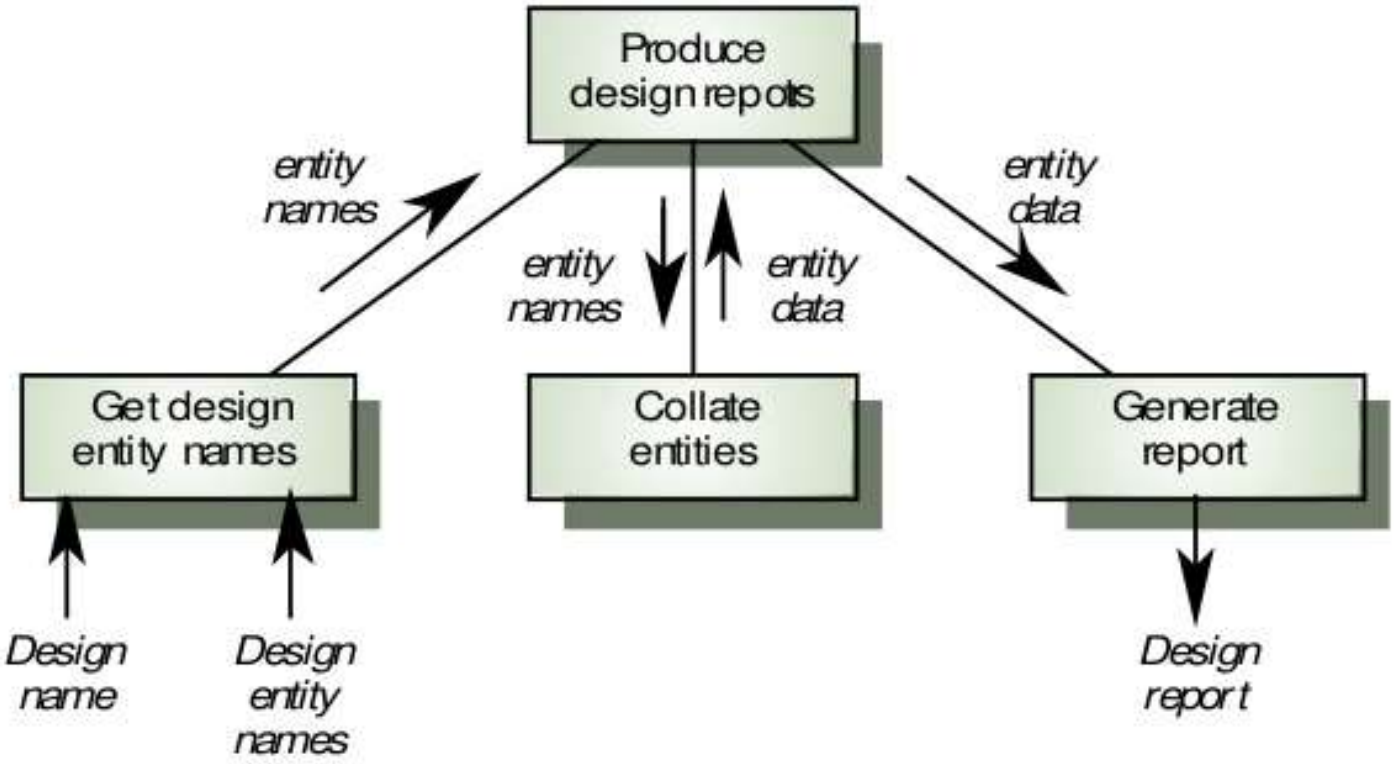
Top-Down Programming (Cont…)



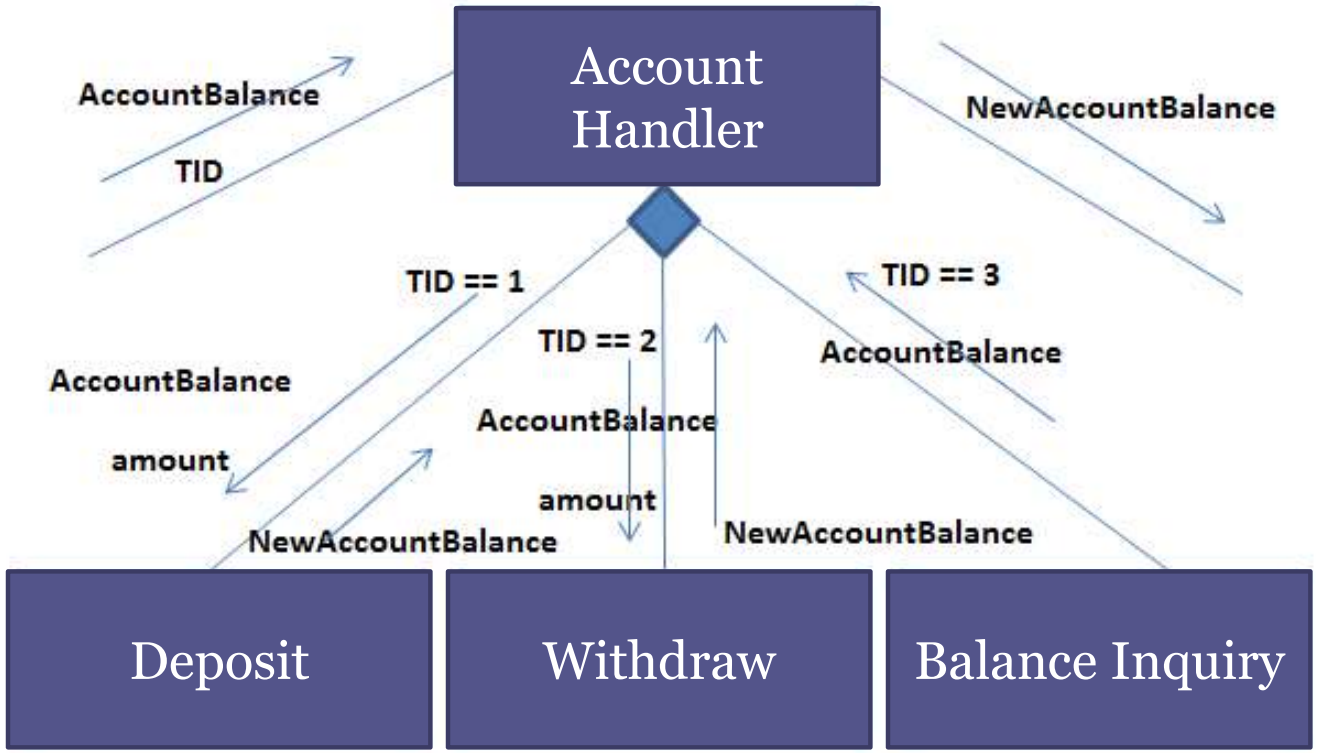
Top-Down Programming (Cont…)



Top-Down Programming (Cont…)



Case Study – Bank System



Case Study – Bank System

- **Input design**

```
int main ()
{
        int accNumber = 0;
        int transactionNo = -1;
        double balance = 0.0;
        printf ("Enter Account Number : ");
        scanf("%i", &accNumber);
        printf ("Get Balance : ");
        scanf("%d", &balance);
        printf ("What you want to do? \n");
        printf (" Deposite   -> 1\n");
        printf (" Withdrawal -> 2\n");
        printf (" Balance inquery    -> 3\n");
        printf ("Enter Your Choice : ");
        scanf("%i", &transactionNo);

        return 0;
}
```

Case Study – Bank System

- **Account Handler Prototype and Initial Structure**

```
void accountHandler(double AccountBalance, int tid)
{
        switch (tid)
        {
                case 1:
                        printf("Deposite\n");
                        break;
                case 2:
                        printf("Withdrawal\n");
                break;
                case 3:
                        printf("Balance         : %0.2d\n",AccountBalance);
                break;
                default:
                        printf ("Wrong choice\n");
                break;
        }
}
```

Case Study – Bank System

- Withdraw Function

```
double withdraw(double AccountBalance, double amount)
{
      printf ("Available balance : %0.2d\n",
                                    AccountBalance);
      printf ("Withdrawal amount : %0.2d\n",
                        amount);
      AccountBalance = AccountBalance - amount;
      return AccountBalance;
}
```

Case Study – Bank System

- Deposit Function

```
double deposit(double AccountBalance, double amount)
{
      printf ("Available balance : %0.2d\n",
                        AccountBalance);
      printf ("Deposite amount   : %0.2d\n",
                        amount);
      AccountBalance = AccountBalance + amount;
      return AccountBalance;
}
```

View Complete Scenario

Recursive Functions

- **Recursion** is a method of solving problems that involves breaking a problem down into smaller and smaller sub-problems until you get to a small enough problem that it can be solved trivially.
- In programming languages, if a program allows you to call a function inside the same function, then it is called a recursive call of the function.

```
void recursion() {
    recursion(); /* function calls itself */
}
int main() {
    recursion();
}
```

Recursive Functions (Cont...)

- **Parts of a Recursive Algorithm**
  - All recursive algorithms must have the following:
    - Base Case (i.e., when to stop)
    - Work toward Base Case
    - Recursive Call (i.e., call ourselves)
  - The "work toward base case" is where we make the problem simpler.
  - The recursive call, is where we use the same algorithm to solve a simpler version of the problem.
  - The base case is the solution to the "simplest" possible problem.

Recursive Functions (Cont…)

- How Recursion Works?
  - In a recursive algorithm, the computer "remembers" every previous state of the problem.
  - This information is "held" by the computer on the **"activation stack"** (i.e., inside of each functions workspace).
  - Every function has its own workspace **PER CALL** of the function.

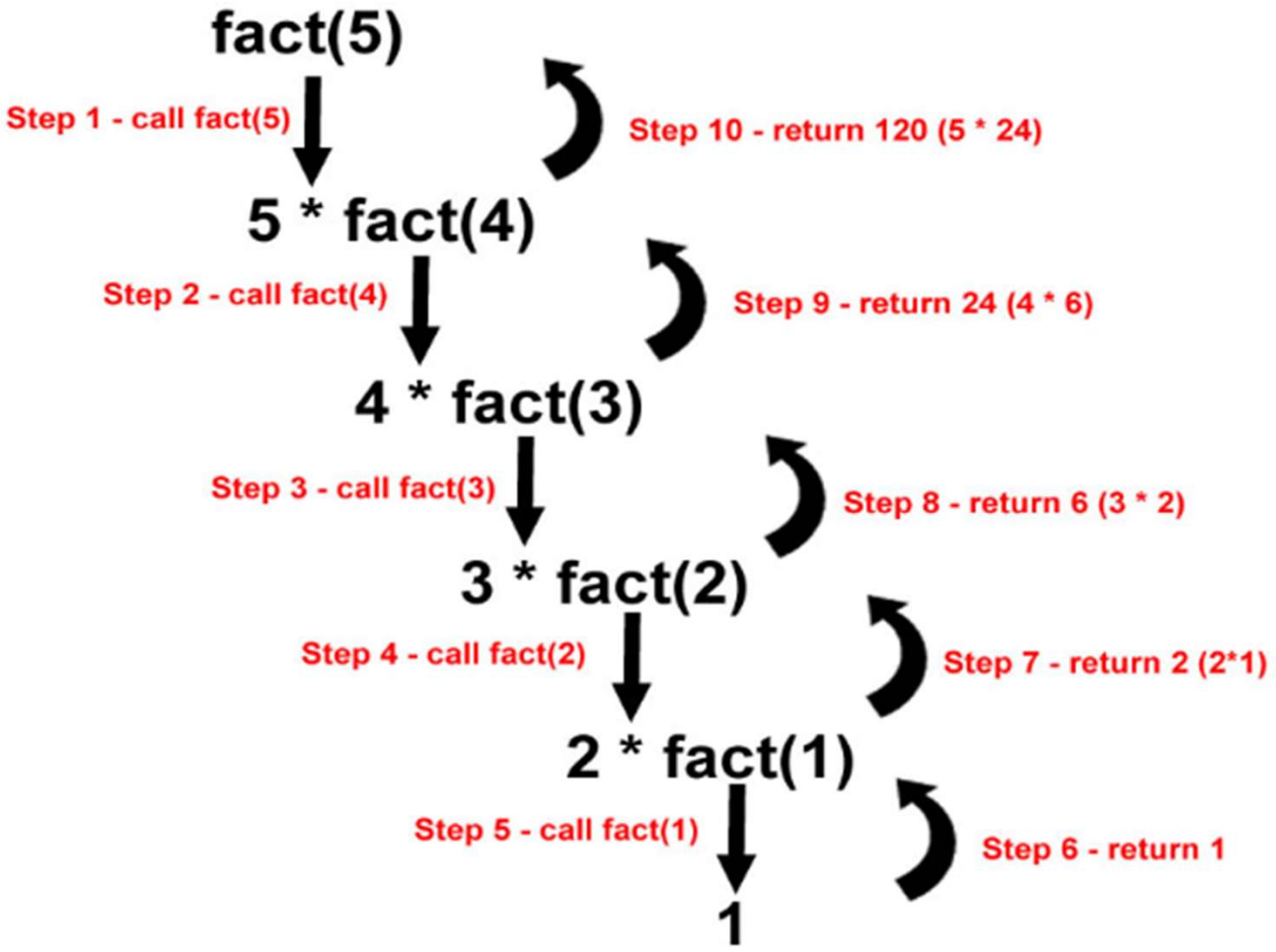
Recursive Functions (Cont…)

- While using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an infinite loop.
- Recursive functions are very useful to solve many mathematical problems, such as calculating the factorial of a number, generating Fibonacci series, etc.

Recursive Functions (Cont…)

```
#include <stdio.h>

int factorial(unsigned int);

int factorial(unsigned int i) {
   if(i <= 1) {
      return 1;
   }
   return i * factorial(i - 1);
}

int  main() {
   int i = 15;
   printf("Factorial of %d is %d\n", i, factorial(i));
   return 0;
}
```

Recursive Functions (Cont…)



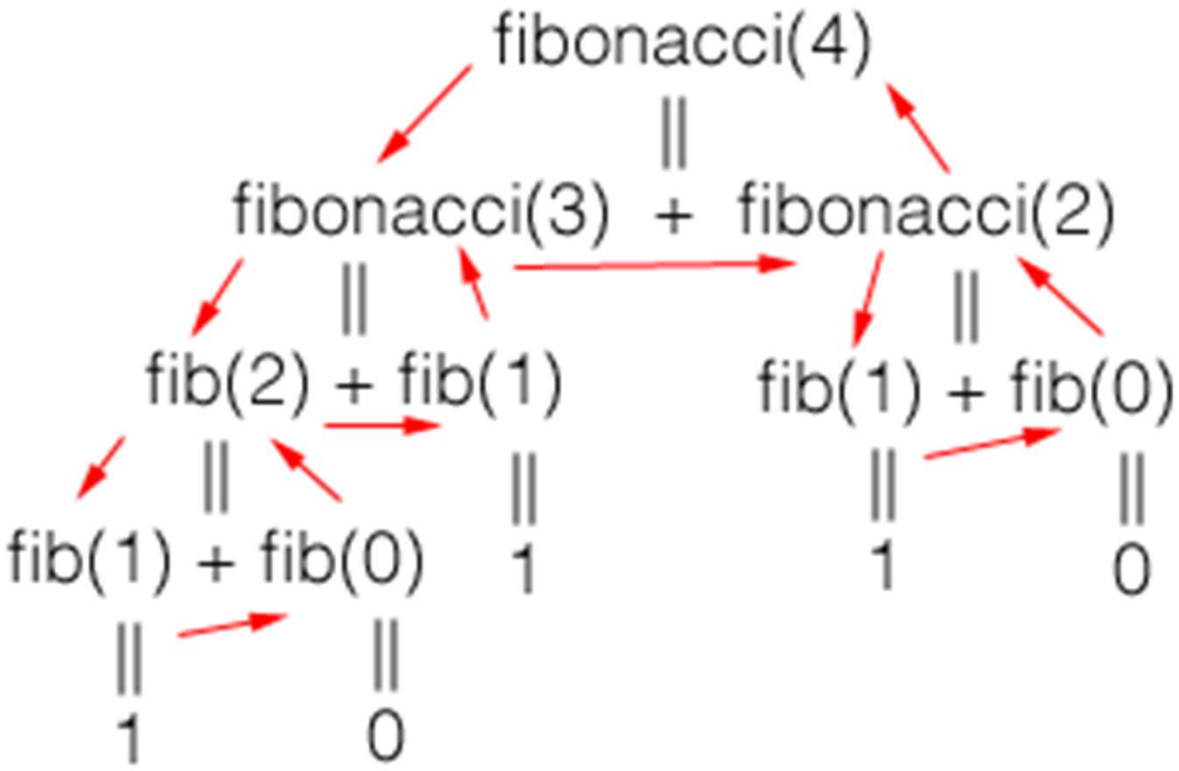
Recursive Functions (Cont…)

```
#include <stdio.h>
int fibonacci(int);

int fibonacci(int i) {

   if(i == 0) {
      return 0;
   }

   if(i == 1) {
      return 1;
   }
   return fibonacci(i-1) + fibonacci(i-2);
}
```

Recursive Functions (Cont…)

```
int  main() {
    int i;
    for (i = 0; i < 10; i++) {
       printf("%d\t\n", fibonacci(i));
    }
    return 0;
}
```

Recursive Functions (Cont…)



Recursive Functions (Cont…)

- Advantages of Recursion
  - Reduce unnecessary calling of function.
  - Through Recursion one can Solve problems in easy way while its iterative solution is very big and complex. For example to reduce the code size for Tower of Honai application, a recursive function is best suited.
  - Extremely useful when applying the same solution.

Recursive Functions (Cont...)

- Disadvantages of Recursion
  - Recursive solution is always logical and it is very difficult to trace.(debug and understand).
  - In recursive we must have an if statement somewhere to force the function to return without the recursive call being executed, otherwise the function will never return.
  - Recursion takes a lot of stack space, usually not considerable when the program is small and running on a PC.
  - Recursion uses more processor time.

Objective Re-cap

- Now you should be able to:
  - Describe the C functions.
  - Practice the declaration, parameter passing, prototyping and calling functions.
  - Justify parameter passing methods.
  - Practice creating user defined C header files.
  - Justify the variable types and scope rules.
  - Describe the concept of recursion.
  - Apply taught concepts for writing programs.

References

- Chapter 08, - Programming in C, 3rd Edition, Stephen G. Kochan

- Acknowledgement
  - https://www.tutorialspoint.com/

Q & A

Next: Input and Output Functions